Inheritance

```
scala> class Animal {
     |   def sound(): Unit = {
     |     println("Animal makes a sound")
     |   }
     | }
class Animal

scala>

scala> class Dog extends Animal {
     |   def bark(): Unit = {
     |     println("Dog barks")
     |   }
     | }
class Dog

scala> var d = new Dog
var d: Dog = Dog@40ddf339

scala> d.bark()
Dog barks

scala> var a = new Animal()
var a: Animal = Animal@2346f77a
scala> a.sound()
Animal makes a sound
```

Polymorphism (method overriding)

```
scala> class Animal{
    | def call():Unit = {println("Animal calling")}}
class Animal

scala> class Dog extends Animal{
    | override def call():Unit = {println("Dog calling")}}
class Dog
scala> var a  = new Animal
var a: Animal = Animal@140fa482

scala> a.call()
Animal calling

scala> var d = new Dog
var d: Dog = Dog@77eb76f

scala> d.call()
Dog calling
```

Abstract class

```
scala> abstract class Animal{
    | def makesound():Unit
    | def sleep():Unit = {println("animal sleeping")}}
class Animal

scala> class Dog extends Animal{
    | def makesound():Unit = {println("bark")}}
class Dog
scala> var a = new Animal
           ^
      error: class Animal is abstract; cannot be instantiated
scala> var d = new Dog
var d: Dog = Dog@2f0e7fa8

scala> d.makesound()
bark

scala> d.sleep()
animal sleeping
```

Primary constructor

```
scala> class parent(val name : String,val age : Int){
     | def display():Unit = {println("Name : "+name+" Age :
"+age.toString())
     | }}
class parent
scala> var james = new parent("james",40)
var james: parent = parent@10f60e36
scala> james.display()
Name : james Age :  40
```